

Although multiple-class models are more useful and natural for describing workloads of real systems, they present problems to the modeler. For instance, it is difficult to obtain parameters (e.g., multiclass service demands, multiclass visit ratios, multiclass multiprogramming levels) for models with multiple classes. Usually, monitoring tools do not provide measurements on a per-class basis. Inferences (sometimes wild guesses) have to be made to parameterize each workload class and to apportion the system overhead among the classes. As a result, it is more difficult to obtain accurate parameters for multiple-class models than for single-class ones. As will be seen in Chap. 10, the process of validation and calibration of multiple-class models is also difficult.

6.2 SIMPLE TWO-CLASS MODEL

Consider a transaction processing system with a processor (CPU) and two disks. The load on the system consists of two types of transactions: queries and updates. During the peak hours, the system is under heavy load, such that four transactions are in memory almost all the time. In an attempt to optimize the average response time, the support analyst might specify that the best execution mix is a combination of three query transactions and one update transaction. The capacity planner wishes to investigate the system performance that would result from possible modifications of the system. Because of the type of workload (i.e., transactions), it would be natural to represent the system as an open model with a maximum multiprogramming level of four. In this case, if a transaction arrives at the system and finds four other transactions already in memory, it has to queue up for memory. Thus, it is necessary to represent the effects of memory constraints, which makes the model non product-form. An alternative view is that of a closed model with a constant number of four transactions in execution. This view is both a good representation for the system during peak hours and does not violate product-form assumptions. The drawback is that the model represents only the time a transaction spends in execution. Memory contention is not captured.

The two types of transactions differ in both the functions they perform and their patterns of resource usage. Update transactions demand more resources than query transactions. It is natural for the analyst to decide to represent the system as a two-class model, as illustrated in Fig. 6.1; one class for query transactions and another class for update transactions. Suppose that the measurement data of Table 6.1 have been collected for the transaction processing system. From Table 6.1 note that the two classes are characterized by different sets of service demands. For example, while update transactions divide their I/O load among the two disks, query transactions do not make use of disk 2 (i.e., service demand at disk 2 is equal to 0). Since the transactions of

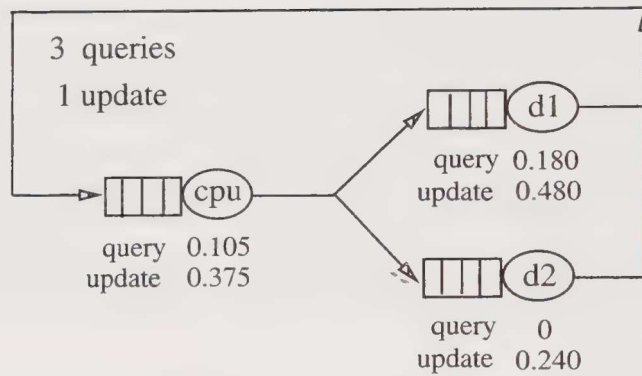


Figure 6.1 Two-class model.

Table 6.1 Measurement Data for the Transaction Processing System.

Class	Service Demand (sec)			Multiprogramming Level	Transactions Completed	Monitoring Period (sec)
	Processor	Disk 1	Disk 2			
Query	0.105	0.180	0	3	7368	1800
Update	0.375	0.480	0.240	1	736	1800

both classes share the same devices, it is reasonable to assume that the service times are roughly the same across the two classes. Thus, the differences in service demands at a given device stem from the number of visits that a transaction makes to the device. In this example, the service demands show that an update transaction performs many more I/O operations than does a query transaction, (i.e., updates require more visits to the disks).

In a single-class model, customers are assumed to be statistically identical in terms of their resource requirements. As a result, customers are undistinguishable in single-class models and the order in which transactions are serviced at a device is irrelevant. This observation, however, is not valid for multiclass models, because customers of different classes are distinct. Therefore, the issue of scheduling is relevant in models of multiple classes. Suppose that at a given instant of time, the four transactions of the example are contending for processor time (i.e., all transactions are at the CPU). A natural question is: Which transaction (query or update) will be serviced next? The goal of a *scheduling policy* is to assign customers to be executed by a server to optimize some system objective, such as minimizing average response time or maximizing throughput. For instance, by specifying a priority scheduling policy, one could force a processor to spend most of its time on classes of customers that are considered critical. In the same way, one could define a scheduling discipline that gives priority to short transactions over large batch jobs, in the attempt of minimizing average response time of short requests.

Actual operating systems implement versions of well-known scheduling disciplines such as *first come first served (FCFS)*, *round robin (RR)*, *shortest job first*, and *shortest remaining time*. In short, multiclass models must specify their assumed scheduling disciplines.

Once the usefulness for multiclass models is recognized, an appropriate solution technique is needed. That is, a solution technique is required that calculates performance measures for each workload class in the model. Making use of methods learned so far, a first approach to solving a multiclass model is to construct an equivalent single-class model. To do that, it is necessary to aggregate the performance parameters of the multiple classes into a single class. The service demand of the aggregate class at any device is the average of the individual classes weighted by the relative class throughputs, as shown in Chap. 3. The degree of multiprogramming of the aggregate class corresponds to the sum of the multiprogramming levels of each component. If the measured throughput of the two classes are $7368/1800 = 4.093$ tps and $736/1800 = 0.409$ tps, respectively, then the processor service demand of the aggregate class is calculated as follows:

$$D_{\text{processor, aggr}} = \frac{0.105 \times 4.093 + 0.375 \times 0.409}{4.502} = 0.1295 \text{ sec}$$

Table 6.2 summarizes the parameters obtained for the aggregate class. The single-class model defined by the parameters of Table 6.2 can then be solved by the techniques introduced in Chap. 5. The calculated throughput for the single-class model equals 4.49, which is worse than its multiclass counterpart, 4.502. Dowdy et al. [1] have shown that a single-class model of an actual multiclass system pessimistically bounds the performance of the multiclass system. These bounds can help the analyst to identify errors that come from an incorrect workload characterization of a multiclass system. Using operational relationships ($U_i = X_0 \times D_i$ and $R = n/X_0$), the following are results for the single-class model: $R = 4/4.49 = 0.891$, $U_{\text{cpu}} = 0.130 \times 4.49 = 58.37\%$, $U_{\text{disk 1}} = 0.207 \times 4.49 = 92.94\%$, and $U_{\text{disk 2}} = 0.022 \times 4.49 = 9.87\%$.

Once the equivalent single-class model has been built and solved, the analyst often wants to use it to investigate the effects of possible future modifications to the system. For example,

- What is the predicted increase in the throughput of query transactions if the load of the update class is moved to off-peak hours?

Table 6.2 Parameters for the Aggregate Class.

Class	Service Demand (sec)			Multiprogramming Level
	Processor	Disk 1	Disk 2	
Aggregate	0.130	0.207	0.022	4

- Realizing that disk 1 is the bottleneck (i.e., the device with the highest utilization) and disk 2 is lightly loaded, what is the predicted response time if the total I/O load of query transactions is moved to disk 2?

Clearly, the equivalent single-class model is not able to answer these questions, because it does not provide performance results on an individual class basis. Therefore, techniques to calculate the performance of models with multiple classes are needed.

6.3 NOTATION AND ASSUMPTIONS

When customers in a queuing network model exhibit different routing patterns and different service time requirements, the model is said to have multiple classes of customers. Consider, for instance, a local area network with P workstations and Q file servers. Each workstation may have its own local storage, but it also requires storage services from the servers. After executing in the workstation, a job goes through the network to require service from the file servers. Each workstation places one job request at a time on the network. A queuing model of this system is proposed by Woodside and Tripathi [2]. If all the workstations have identical workload requirements and assign their services to all file servers, the system can be modeled by a single-class workload with P jobs. However, since each job has a different routing probability (i.e., each job visits a different set of servers), a separate workload class is required for each workstation. Thus, the system is modeled with P classes; each with exactly one job.

The queuing networks considered here consist of K devices (or service centers) and R different classes of customers. A central concept in the analysis and solution of queuing networks is the state of the network. The state represents a distribution of customers over classes and devices. The network state is denoted by a vector $\vec{n} = (\vec{n}_1, \vec{n}_2, \dots, \vec{n}_K)$, where component $\vec{n}_i$ ($i = 1, \dots, K$) is a vector that represents the number of customers of each class at server i . That is, $\vec{n}_i = (n_{i,1}, n_{i,2}, \dots, n_{i,R})$. For instance, the state of the transaction system example is defined as $(\vec{n}_1, \vec{n}_2, \vec{n}_3)$, where $\vec{n}_i = (\text{update}, \text{query})$ specifies the number of update and query transactions at device i . Possible states of the transaction system example include $((1, 3), (0, 0), (0, 0))$ and $((0, 1), (0, 2), (1, 0))$. Whereas the former indicates that all transactions are at the processor, the latter represents the situation where one query transaction is executing at the processor, two queries are at disk 1, and the update transaction is being serviced at disk 2.

The BCMP theorem [3], developed by Baskett, Chandy, Muntz, and Palacios, specifies the combination of service time distributions and scheduling disciplines that yield multiclass product-form queuing networks. Open, closed, or mixed networks are allowed. A closed class is one in which a

constant number of customers remain in the network at all times. In contrast, in an open class, customers are allowed to enter or leave the network. A mixed network is closed with respect to some classes and open with respect to other classes. Basically, the set of assumptions required by the BCMP theorem for a product-form solution is as follows.

- *Service centers with a FCFS discipline.* In this case, customers are serviced in the order in which they arrive. The service time distributions are required to be exponential with the same mean for all classes. Although all classes must have the same mean service time at a given device, they may have different visit ratios, which means the possibility of different service demands for each class at a given device. The service rate is allowed to be load dependent, but it can be dependent only on the total number of customers at the server and not on the number of customers of any particular class. For instance, service requests from workstations to a file server can be adequately modeled by the FCFS discipline. Disks are usually modeled as FCFS service centers.
- *Service centers with a PS discipline.* When there are n customers at a server with a processor sharing discipline, each customer receives service at a rate of $1/n$ of their normal service rate. Each class may have a distinct service time distribution. The *round robin* (RR) scheduling discipline allows customers to receive a fixed quantum of service. If the customer does not finish execution within its allocated quantum, it returns to the end of the queue, awaiting further service. In the limiting case, when the quantum approaches zero, the RR discipline becomes the PS discipline. Time-sharing operating systems such as UNIX employ the RR discipline to processor scheduling. Thus, PS is a reasonable approximation to represent processor scheduling disciplines of actual operating systems.
- *Service centers with infinite servers (IS).* When there is an infinite supply of servers in a service center, there is never any waiting for a server. This situation is known as IS, delay server, or no queuing. For instance, think time at terminals in an interactive system is usually modeled by delay servers. In a lightly loaded LAN, the time needed to send a request from a workstation to a server can also be approximated by a delay server.
- *Service centers with a LCFS-PR discipline.* Under last come first served preemptive resume, whenever a new customer arrives, the server ceases (i.e., preempts) servicing the previous customer (if any) and proceeds to serve the new arrival. When a customer completes, the server resumes executing its previously preempted job. Each class may have a distinct service time distribution.